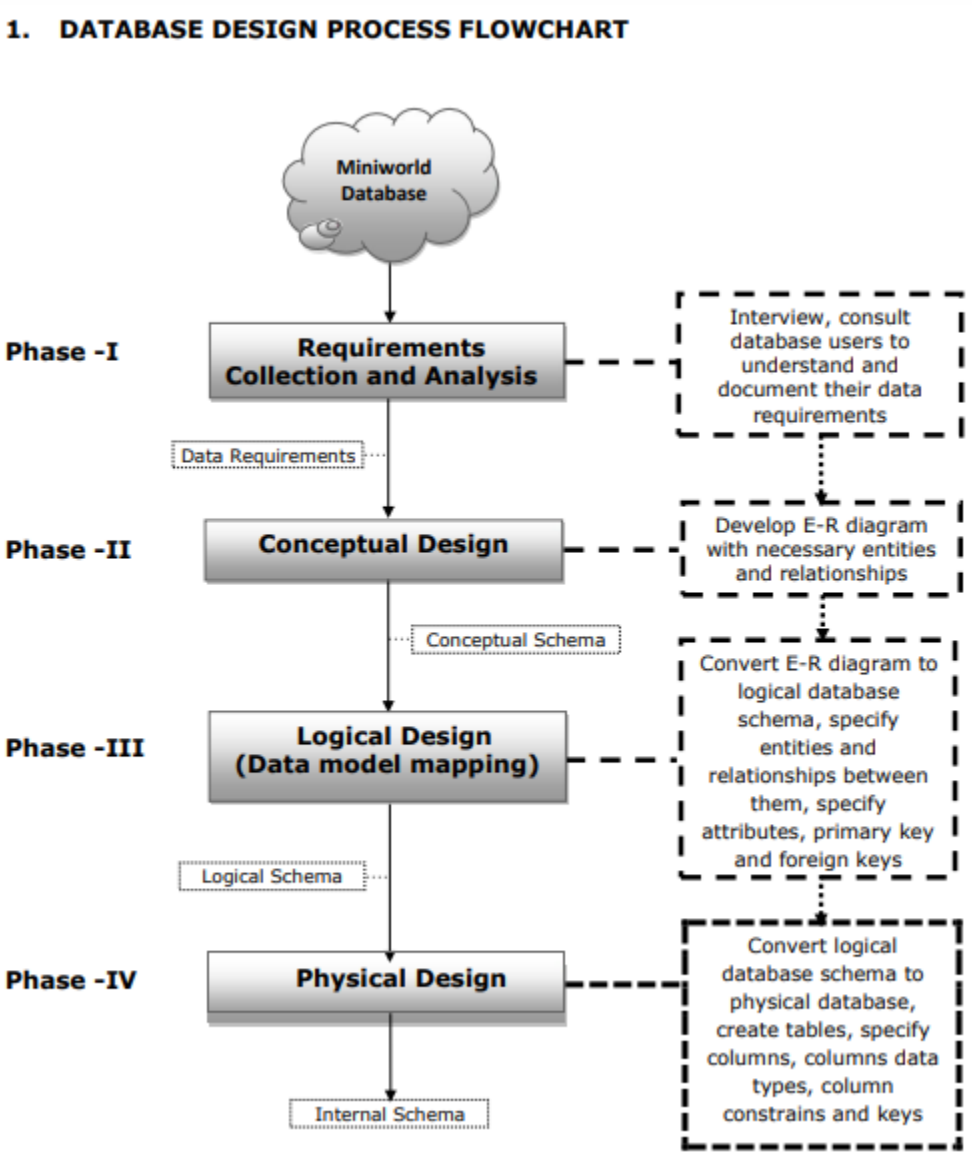
Structured Query Language part 1 (Physical schema implementation)



Database Normalization:

Normalization is a systematic process in database systems used to organize data to reduce redundancy and improve data integrity. It involves dividing a database into tables and defining relationships between them to minimize duplicate data, eliminate update anomalies, and ensure data dependencies make sense.

Purpose of Normalization

Normalization is used to:

1. **Reduce Data Redundancy:** Prevent duplicate data across tables, which saves storage and minimizes the risk of inconsistencies.
2. **Enhance Data Integrity:** Ensure that data is accurate, consistent, and adheres to defined constraints.
3. **Simplify Data Maintenance:** With reduced redundancy, it's easier to update, insert, delete, or retrieve data without affecting other tables.

Types of Normal Forms

Normalization involves several levels or "normal forms," each with specific rules:

1. First Normal Form (1NF)

- **Rule:** Eliminate repeating groups in individual tables. Ensure each field contains only atomic (indivisible) values and each record is unique (No multi-value attributes)
- **Example:** Consider a Students table where each student may be enrolled in multiple courses listed in a single column.

  StudentID | StudentName | Courses

  ----------|-------------|-----------------

  1 | Alice | Math, Science

  2 | Bob | English, History, Math

- **Conversion to 1NF:** Each course must be in a separate row.

  StudentID | StudentName

  ----------|-------------|-----------------

  1 | Alice

  2 | Bob

  StudentId|CourseNme

  ----------|-------------|-----------------

  1 | Math

  1| Science

  2|English

---

2. Second Normal Form (2NF)

- **Rule:** Achieve 1NF and ensure that all non-key attributes are fully functionally dependent on the primary key (eliminate partial dependency).
- **Example:** A CourseEnrollments table where the primary key is a composite key of StudentID and CourseID, and StudentName is a partial dependency.

  StudentID | CourseID | StudentName | CourseName

  ----------|----------|-------------|-----------

  1 | 101 | Alice | Math

  1 | 102 | Alice | Science

  2 | 101 | Bob | Math

- **Conversion to 2NF:** Remove the partial dependency by separating the StudentName into a Students table.

  Students:

  StudentID | StudentName

  ---------|-----------

  1 | Alice

  2 | Bob

  Courses:

  CourseID | CourseName

  ---------|-----------

  101 | Math

  102 | Science

  CourseEnrollments:

  StudentID | CourseID

  ---------|-----------

  1 | 101

  1 | 102

  2 | 101

3. Third Normal Form (3NF)

- **Rule:** Achieve 2NF and remove transitive dependencies, where non-key attributes depend on other non-key attributes.
- **Example:** A Orders table where OrderID is the primary key, CustomerID is a foreign key, and CustomerAddress depends on CustomerID but not directly on OrderID.

  OrderID | CustomerID | CustomerName | CustomerAddress

  --------|------------|--------------|-----------------

  1 | 100 | Alice | 123 Main St

  2 | 101 | Bob | 456 Maple Ave

- **Conversion to 3NF:** Separate CustomerName and CustomerAddress into a Customers table to eliminate transitive dependencies.

  Orders:

  OrderID | CustomerID

  --------|------------

  1 | 100

  2 | 101

  Customers:

  CustomerID | CustomerName | CustomerAddress

  -----------|--------------|-----------------

  100 | Alice | 123 Main St

  101 | Bob | 456 Maple Ave

---

4. Boyce-Codd Normal Form (BCNF)

- **Rule:** A stronger version of 3NF where, in addition to 3NF, every determinant is a candidate key. It eliminates certain types of anomalies not addressed in 3NF.
- **Example:** Similar to 3NF but ensures even stricter adherence by ensuring all determinants are keys.

5. Fourth Normal Form (4NF)

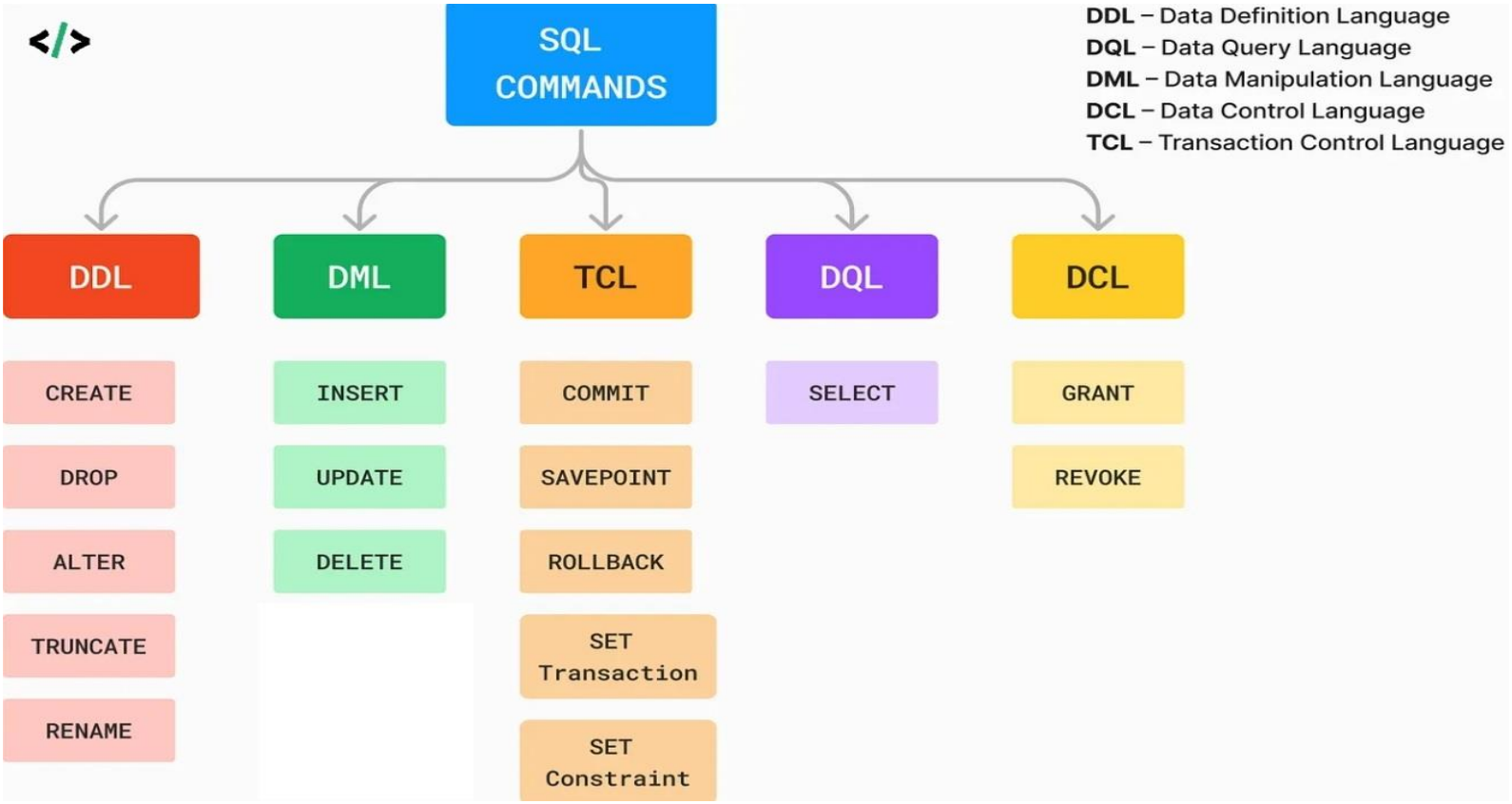
- **Rule:** Achieve BCNF and remove multi-valued dependencies where one attribute depends on multiple independent attributes.
- **Example:** If a table contains StudentID, Course, and Hobby, but Course and Hobby are independent, separate them into different tables.

6. Fifth Normal Form (5NF)

- **Rule:** Decompose tables to eliminate join dependencies, ensuring no loss of data and avoiding spurious tuples upon rejoining.
- **Example:** Generally used for complex relationships where 3NF, BCNF, and 4NF do not resolve issues related to dependency preservation.

SQL (Structured Query Language)

- **Definition:** SQL is a standardized programming language used for managing and manipulating relational databases. It allows users to perform various operations, such as querying data, updating records, inserting new records, and deleting records.
- **Purpose:** SQL is used to communicate with databases and perform tasks such as:
    - **Data Retrieval -- DQL:** Using SELECT statements to retrieve specific data from tables.
    - **Data Manipulation -- DML:** Using INSERT, UPDATE, and DELETE statements to modify data.
    - **Data Definition -- DDL:** Using CREATE, ALTER, and DROP statements to define or modify database structures, such as tables and indexes.
    - **Data Control -- DCL:** Using GRANT and REVOKE statements to control access to database objects.
    - **Transactions Control –** TCL : For achieving batch process as atomic



2. SQL Server

- **Definition:** SQL Server is a relational database management system (RDBMS) developed by Microsoft. It supports storing, managing, and retrieving data for various types of applications.

3. SQL Server Management Studio (SSMS)

- **Definition:** SSMS is a graphical user interface (GUI) tool provided by Microsoft for managing SQL Server instances and databases. It is an integrated environment for working with SQL Server databases, providing tools for database administration, development, and querying.

In summary:

- **SQL** is the language used to interact with databases.
- **SQL Server** is a database management system that stores, manages, and retrieves data.
- **SQL Server Management Studio (SSMS)** is a tool for managing SQL Server instances and

1. Data Definition Language (DDL)

In SQL, **Data Definition Language (DDL)** commands like **CREATE**, **ALTER**, **DROP**, and **TRUNCATE** are used to define and manage the structure of database tables. Additionally, **integrity constraints** like domain, entity, and referential constraints are critical for maintaining data accuracy and consistency. Here's a full explanation of these DDL commands and integrity constraints, complete with examples.

1. CREATE Database

- **Purpose:** The CREATE DATABASE statement is used to create a new database in SQL. This command sets up an empty database that you can then populate with tables, views, and other database objects.
- **Syntax:**

  CREATE DATABASE DatabaseName;

Example:

  CREATE DATABASE Company;

---

2. CREATE TABLE

- **Purpose:** The CREATE TABLE command is used to define a new table and specify its columns, data types, and any constraints.
- **Syntax:**

```
CREATE TABLE TableName (
  Column1 DataType [Constraint],
  Column2 DataType [Constraint],
  ...
);
```

Example:

```
CREATE TABLE Employees (
  EmployeeID INT PRIMARY KEY,
  FirstName VARCHAR(50) NOT NULL,
  LastName VARCHAR(50) NOT NULL,
  Department VARCHAR(50),
  Salary DECIMAL(10, 2) CHECK (Salary > 0)
);
```

Explanation: This example creates an Employees table with columns for EmployeeID, FirstName, LastName, Department, and Salary. The EmployeeID is a primary key, and there is a check constraint ensuring Salary is positive.

---

3. ALTER TABLE

- **Purpose:** The ALTER TABLE command is used to modify the structure of an existing table, such as adding, modifying, or dropping columns, or adding constraints.
- **Syntax:**

```
ALTER TABLE TableName
ADD ColumnName DataType [Constraint];

ALTER TABLE TableName
MODIFY ColumnName NewDataType;

ALTER TABLE TableName
DROP COLUMN ColumnName;
```

Examples:

```
-- Adding a new column to the table
ALTER TABLE Employees
ADD DateOfBirth DATE;

-- Modifying the data type of an existing column
ALTER TABLE Employees
MODIFY Salary DECIMAL(12, 2);

-- Dropping a column from the table
ALTER TABLE Employees
DROP COLUMN Department;
```

Explanation: In these examples, the ALTER TABLE command is used to add a DateOfBirth column, modify the Salary column to allow a larger range, and remove the Department column.

---

4. DROP TABLE

- **Purpose:** The DROP TABLE command is used to delete an entire table, including all of its data, structure, and associated constraints.
- **Syntax:**

```
DROP TABLE TableName;
```

- **Example:**

```
DROP TABLE Employees;
```

Explanation: This command completely removes the Employees table from the database. **Be cautious** when using DROP TABLE as it is irreversible and deletes all data within the table permanently.

---

5. TRUNCATE TABLE

- **Purpose:** The TRUNCATE TABLE command is used to delete all rows from a table without removing the table structure itself. Unlike DELETE, it cannot be rolled back as it does not log individual row deletions.
- **Syntax:**

```
TRUNCATE TABLE TableName;
```

- **Example:**

```
TRUNCATE TABLE Employees;
```

Explanation: This command removes all records from the Employees table, but the table structure remains intact, allowing new data to be inserted. It is faster than DELETE for large tables.

Integrity Constraints

Integrity constraints are rules applied to columns in a table to ensure data validity and consistency. There are three main types:

1. Domain Integrity

- **Purpose:** Ensures that data in a column falls within a specific range, type, or set of permissible values. This is enforced through data types, default values, and constraints like CHECK and UNIQUE,
- **Example:**

```
CREATE TABLE Products (
  ProductID INT PRIMARY KEY,
  ProductName VARCHAR(100) NOT NULL UNIQUE,
  Price DECIMAL(10, 2) CHECK (Price > 0),
  Category VARCHAR(50) DEFAULT 'General'
);
```

Explanation: In this Products table, Price has a CHECK constraint that enforces domain integrity by ensuring that the price is always greater than zero. The Category column has a default value of 'General', which applies when no value is specified, further enforcing domain constraints.

---

2. Entity Integrity

- **Purpose:** Ensures that each row in a table is uniquely identifiable, typically enforced through primary keys. No two rows can have the same primary key value, and primary keys cannot be null.
- **Example:**

```
CREATE TABLE Customers (
  CustomerID INT PRIMARY KEY,
  FirstName VARCHAR(50) NOT NULL,
  LastName VARCHAR(50) NOT NULL,
  Email VARCHAR(100) );
```

Explanation: The Customers table has CustomerID as a primary key, ensuring that each customer has a unique identifier. Additionally, the Email column has a UNIQUE constraint, preventing duplicate email addresses in the table.

---

3. Referential Integrity

- **Purpose:** Maintains consistency between tables by ensuring that foreign keys in a child table match primary keys in a related parent table. Referential integrity ensures that relationships between tables remain valid.
- **Example:**

```
CREATE TABLE Orders (
  OrderID INT PRIMARY KEY,
  OrderDate DATE NOT NULL,
  CustomerID INT,
  FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)
);
```

Explanation: In the Orders table, CustomerID is a foreign key referencing the CustomerID primary key in the Customers table. This relationship enforces that any CustomerID in Orders must already exist in Customers. If you try to insert an order with a CustomerID that doesn't exist in Customers, the database will prevent it.

Why Integrity Constraints Matter

Integrity constraints are crucial in database systems because they:

- **Ensure Data Accuracy:** Prevent invalid or inconsistent data entries.
- **Maintain Relationships:** Enforce the logical connections between tables.
- **Improve Data Quality:** Guarantee that only valid data gets stored.
- **Prevent Anomalies:** Minimize issues like duplicate entries or orphaned records.

Using these DDL commands and applying integrity constraints helps design robust, efficient, and reliable databases that maintain data integrity over time.
